The Krasnosel'skiĭ-Mann Iterative Method

Chapter 6: The Multi-step Inertial Krasnosel'skiĭ-Mann Iteration

Based on Dong, Cho, He, Pardalos & Rassias

Table of Contents

Notation You'll Need / Recap I

Setting. $\mathcal{H}$ is a real Hilbert space, $T : \mathcal{H} \to \mathcal{H}$ is *nonexpansive* ($\|Tx - Ty\| \leq \|x - y\|$) with $\mathrm{Fix}(T) = \{x : Tx = x\} \neq \emptyset$.

Recap: plain Krasnosel'skiĭ–Mann (KM) iteration

$$x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T(x_n), \qquad n \geq 0,$$

where $\lambda_n \in [0, 1]$. If $\sum_n \lambda_n(1 - \lambda_n) = \infty$, then $x_n \rightharpoonup$ (weakly converges to) a point of $\mathrm{Fix}(T)$.

Recap: single-step (general) inertial KM – Chapter 5

$$\begin{cases} y_n = x_n + a_n(x_n - x_{n-1}), \\ z_n = x_n + b_n(x_n - x_{n-1}), \\ x_{n+1} = (1 - \lambda_n)y_n + \lambda_n T(z_n), \end{cases}$$

where $a_n, b_n \in (-1, 2]$ are *inertial parameters*. Only **one** lagged difference, $x_n - x_{n-1}$, is used – like remembering only your most recent velocity. Setting $a_n = b_n$ recovers the *classical* inertial KM iteration.

Notation You'll Need / Recap II

Intuition: from one step of memory to many. A single-step inertial method is a ball that remembers its *last* velocity and keeps rolling a bit further in that direction before the next correction. A **multi-step** inertial method is a heavier ball with a longer memory: it looks back at its last s moves and adds a *weighted combination* of all of them, not just the most recent one. Slowly-decaying oscillations in the trajectory can be damped out by mixing in older velocities with smaller weights, while a persistent drift direction gets reinforced across several lagged terms at once.

The multi-step inertial term (general form)

For $s \geq 1$ and a window $S_n \subseteq \{0, 1, \ldots, n-1\}$ of lag indices,

$$\sum_{k \in S_n} a_{n,k} \, (x_{n-k} - x_{n-k-1}).$$

Symbols. k indexes *how far back* we look ($x_{n-k} - x_{n-k-1}$ is the k-lag "velocity"); S_n is the set of lags actually used at step n (e.g. $S_n = \{0, 1\}$ uses the two most recent velocities); $a_{n,k}$ is the weight given to the k-lag velocity at time n. Taking $S_n \equiv \{0\}$ collapses this back to the single-step term $a_n(x_n - x_{n-1})$.

Notation You'll Need / Recap IV

What "does not depend on the iterative sequence" means

A parameter sequence $\{a_{n,k}\}$ is said *not to depend on* $\{x_n\}$ if you could write down its numerical values **before ever running the algorithm** – e.g. $a_{n,k} = (1-\mu)\mu^k$ for a fixed constant μ. In plain English: you can *pre-schedule* these numbers on a piece of paper in advance, rather than watching the run unfold and *adapting* the weights to what you observe (e.g. clipping $a_{n,k}$ based on the size of $\|x_{n-k} - x_{n-k-1}\|$ you just measured).

- **Depends on** $\{x_n\}$ ("conditional"): $a_{n,k}$ is a function of quantities computed *during* the run, such as $\|x_{n-k} - x_{n-k-1}\|$.
- **Does not depend on** $\{x_n\}$ ("unconditional"): $a_{n,k}$ is a deterministic function of n, k only, fixed once and for all.

This distinction drives the whole chapter: Section 6.1 gives the most general multi-step scheme (whose convergence proof needs a condition that, in general, *does* involve $\{x_n\}$); Section 6.2 exhibits two concrete parameter choices that are pre-scheduled and therefore need no run-time bookkeeping at all.

Notation You'll Need / Recap V

Running example (carried through this chapter). $\mathcal{H} = \mathbb{R}^2$, $T = P_C =$ projection onto the closed unit disk $C = \{x : \|x\| \leq 1\}$, started at $x_0 = (3, 4)$. Since $x_0/\|x_0\| = (0.6, 0.8)$ and every KM-type map studied below moves *along the ray through the origin and* x_0, the iterates in this example converge to the boundary point $(0.6, 0.8)$.

6.1 Intuition

Why go beyond one lag? Liang's Ph.D. thesis and Ortega–Rheinboldt's classical "s-step methods" both observed that using *several* previous iterates $x_n, x_{n-1}, \ldots, x_{n-s+1}$ to build the next one can improve convergence – extrapolating not just from the last displacement but from a whole recent trajectory segment. Dong et al. (2021) imported this idea directly into the KM iteration, producing the *multi-step inertial KM iteration*: the most general inertial KM scheme in the literature, of which *every* scheme from Chapter 5 (original, reflected, accelerated, classical inertial, general inertial KM) is a special case.

The Multi-step Inertial KM Iteration (6.5)

Definition

Let $x_0, x_{-1} \in \mathcal{H}$. For each $n \geq 0$, set

$$\begin{cases} y_n = x_n + \displaystyle\sum_{k \in S_n} a_{n,k}(x_{n-k} - x_{n-k-1}), \\ z_n = x_n + \displaystyle\sum_{k \in S_n} b_{n,k}(x_{n-k} - x_{n-k-1}), \\ x_{n+1} = (1-\lambda_n)y_n + \lambda_n T(z_n), \end{cases}$$

where $S_n \subseteq \{0, 1, \ldots, n-1\}$, $a_{n,k}, b_{n,k} \in (-1, 2]^{|S_n|}$, and $|S_n|$ is the number of elements of S_n.

Symbols. y_n, z_n: two (possibly different) inertially-extrapolated points; $\lambda_n \in [0, 1]$: the usual KM averaging parameter; S_n: the active window of lags at step n (its size can even grow with n); $a_{n,k}$ (resp. $b_{n,k}$): the weight on the k-lag velocity feeding into y_n (resp. z_n).

Recovering Everything We Already Know

Fix $S_n \equiv \{0\}$ (single lag) and write $a_n := a_{n,0}$, $b_n := b_{n,0}$.

Choice of a_n, b_n	Recovered scheme
$a_n = 0,\ b_n = 0$	original KM iteration
$a_n \in [0, a],\ b_n = 0$	reflected KM iteration
$a_n = 0,\ b_n \in [0, b]$	accelerated KM iteration
$a_n \in [0, a],\ b_n = a_n$	classical inertial KM (Ch. 5)
$a_n \in [0, a],\ b_n \in [0, b],\ a_n \neq b_n$	general inertial KM (Ch. 5)

So (6.5) with $\{0\} \subset S_n$ (at least one lag > 0 present) is *brand new*: it genuinely uses memory beyond the last displacement.

Convergence: Theorem 6.2 I

Strategy of proof. Rewrite (6.5) as a plain KM iteration with two *perturbations* e_n^1, e_n^2:

$$x_{n+1} = (1-\lambda_n)(x_n+e_n^1)+\lambda_n T(x_n+e_n^2), \quad e_n^1 = \sum_{k\in S_n} a_{n,k}(x_{n-k}-x_{n-k-1}),\ e_n^2 = \sum_{k\in S_n} b_{n,k}(x_{n-k}-x_{n-k-1}).$$

The classical KM-with-perturbations theorem then gives convergence *provided the perturbations are eventually small enough (summable)*.

Theorem 6.2

Let $T : \mathcal{H} \to \mathcal{H}$ be nonexpansive with $\mathrm{Fix}(T) \neq \emptyset$. Assume $\sum_{n=0}^{\infty} \lambda_n(1-\lambda_n) = \infty$ and

$$\sum_{n=0}^{\infty} \max\Big\{ \max_{k\in S_n} |a_{n,k}|,\ \max_{k\in S_n} |b_{n,k}| \Big\} \sum_{k\in S_n} \|x_{n-k} - x_{n-k-1}\| < \infty. \tag{6.9}$$

Then $\{x_n\}$ generated by (6.5) converges weakly to a point of $\mathrm{Fix}(T)$.

Convergence: Theorem 6.2 II

Reading condition (6.9). It bounds a *product*: (the largest inertial weight in use) $\times$ (the total size of the lagged displacements). Either factor being small enough makes the product summable. If $a_{n,k}, b_{n,k} \in [0,1]$, the absolute values simplify away:

Simplified condition – Remark 6.2

$$\sum_{n=0}^{+\infty} \max\left\{ \max_{k\in S_n} a_{n,k},\ \max_{k\in S_n} b_{n,k} \right\} \sum_{k\in S_n} \|x_{n-k} - x_{n-k-1}\| < +\infty. \tag{6.10}$$

A concrete online updating rule – (6.11)

For $a_k, b_k \in [0,1]$ fixed and $c_{a,n,k}, c_{b,n,k} > 0$ chosen so their max times $\sum_{k\in S_n} \|x_{n-k} - x_{n-k-1}\|$ is summable, set

$$a_{n,k} = \min\{a_k,\ c_{a,n,k}\}, \qquad b_{n,k} = \min\{b_k,\ c_{b,n,k}\},$$

e.g. $c_{a,n,k} = \dfrac{c_{a,k}}{n^{1+\delta} \sum_{k\in S_n} \|x_{n-k} - x_{n-k-1}\|}$ for constants $c_{a,k} > 0,\ \delta > 0$ (similarly for $c_{b,n,k}$).

Convergence: Theorem 6.2 III

This is a *safety valve*: use the "preferred" weight a_k unless the observed displacements are too large, in which case shrink the weight just enough to keep (6.10) summable. In practice, with well-chosen a_k, b_k, the clipping in (6.11) is rarely triggered.

Rate Results – Remark 6.3

Beyond weak convergence, one can also get *rates*:

- **Pointwise rate** $O(1/\sqrt{n})$: need $0 < \inf_n \lambda_n \le \sup_n \lambda_n < 1$ and the *stronger* summability

$$\sum_{n=0}^{+\infty} (n+1) \max\left\{ \max_{k \in S_n} |a_{n,k}|, \max_{k \in S_n} |b_{n,k}| \right\} \sum_{k \in S_n} \|x_{n-k} - x_{n-k-1}\| < +\infty.$$

- **Ergodic rate**: an analogous condition (same flavor, applied to the running average of the iterates) yields a rate for the ergodic sequence.

Both can again be enforced with the online rule (6.11) (with different exponents δ), i.e. these rates are also achievable without knowing $\{x_n\}$ in advance – only *while* running the algorithm.

The Special Case $S_n \equiv \{0\}$ and Remark 6.4 I

Set $S_n \equiv \{0\}$, $a_n := a_{n,0}$, $b_n := b_{n,0}$: this is exactly the *general inertial KM iteration* of Chapter 5.

Theorem 6.3 – The Conditional Convergence

Let $T : \mathcal{H} \to \mathcal{H}$ be nonexpansive with $\mathrm{Fix}(T) \neq \emptyset$. Assume $\sum_n \lambda_n(1-\lambda_n) = \infty$ and

$$\sum_{n=0}^{+\infty} \max\{a_n, b_n\} \, \|x_n - x_{n-1}\| < \infty. \tag{6.12}$$

Then $\{x_n\}$ generated by the general inertial KM algorithm weakly converges to a point of $\mathrm{Fix}(T)$.

Why "conditional"? Condition (6.12) mixes the *parameters* a_n, b_n with the *iterates* x_n themselves – you cannot check it, let alone design a_n, b_n to satisfy it, without already knowing how $\|x_n - x_{n-1}\|$ behaves. That is precisely a condition that *depends on the iterative sequence*, in the sense defined earlier.

The Special Case $S_n \equiv \{0\}$ and Remark 6.4 II

Remark 6.4 – the case $b_n = a_n$, spelled out

When $b_n = a_n$ for every $n \geq 1$ (the classical inertial KM iteration), condition (6.12) collapses to a single sum:

$$\sum_{n=0}^{+\infty} a_n \, \|x_n - x_{n-1}\| < +\infty. \tag{$\star$}$$

Step-by-step, what this says. Write $d_n := \|x_n - x_{n-1}\|$, the size of the n-th "step". Condition $(\star)$ demands that the weighted series $\sum a_n d_n$ converge. Two safe regimes:

1. a_n *constant* > 0: then $(\star)$ reduces to $\sum_n d_n < \infty$, i.e. the steps themselves must be absolutely summable (a fairly strong, but often true, requirement near a fixed point).
2. d_n *not obviously summable*: then a_n must be chosen to decay fast enough (e.g. $a_n = O(1/n^{1+\delta})$) to force the product down – exactly the role played by the online rule (6.11).

The book notes this is genuinely *different* from condition (C2) used elsewhere in the literature (which typically bounds $\sum a_n \|x_n - x_{n-1}\|^2$, a squared version) – (6.12)/$(\star)$ is a first-power condition.

The Special Case $S_n \equiv \{0\}$ and Remark 6.4 III

The practical cost. If S_n is large – say $S_n = \{0, \ldots, n-1\}$ so $|S_n| = n$ – then *every* $a_{n,k}, b_{n,k}$ satisfying rule (6.11) must be recomputed at every iteration, an $O(n)$-per-step cost that becomes expensive as n grows. This observation is exactly what motivates Section 6.2: find inertial parameter sequences that *never need to look at* $\{x_n\}$ *at all*.

Worked Example: Setup

Running example. $\mathcal{H} = \mathbb{R}^2$, $C =$ closed unit disk, $T = P_C(x) = x/\|x\|$ if $\|x\| > 1$, else $T(x) = x$. Start $x_0 = (3, 4)$, $\lambda_n \equiv \lambda = 0.5$. Target: $x^\star = (0.6, 0.8) \in \mathrm{Fix}(T)$.

Single-step inertial KM recap (Ch. 5), constant $a = 0.1$

$$y_n = x_n + 0.1\,(x_n - x_{n-1}), \qquad x_{n+1} = (1-\lambda)y_n + \lambda T(y_n).$$

Multi-step inertial KM, $s = 2$, constant weights

Take $S_n = \{0, 1\}$ and $b_{n,k} = a_{n,k}$ (so $z_n = y_n$, Remark 6.4 case), with $\theta_1 = 0.13$ on the 1-lag velocity and $\theta_2 = 0.01$ on the 2-lag velocity:

$$y_n = x_n + \theta_1(x_n - x_{n-1}) + \theta_2(x_{n-1} - x_{n-2}), \qquad x_{n+1} = (1-\lambda)y_n + \lambda\, T(y_n).$$

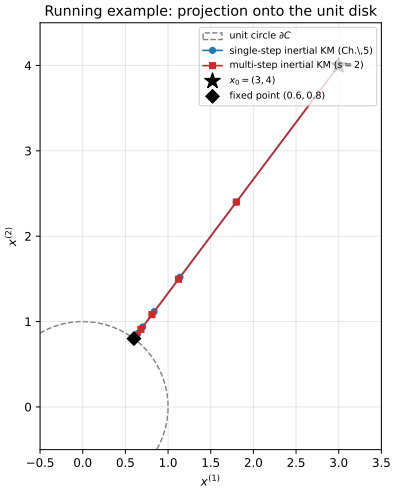
Both are warm-started with $x_{-1} = x_{-2} = x_0$ (all on the ray through the origin and x_0, so the whole trajectory stays on that ray).

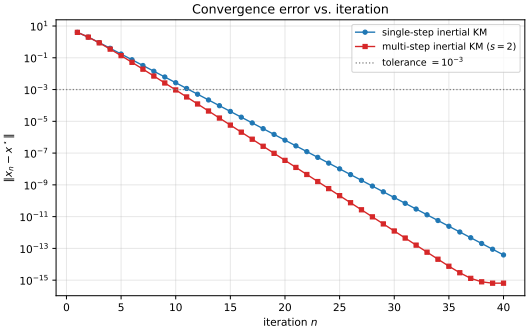
Worked Example: Iterates by Hand

n	single-step x_n	$\|x_n - x^\star\|$	multi-step x_n	$\|x_n - x^\star\|$
1	(3.000000, 4.000000)	4.000000	(3.000000, 4.000000)	4.000000
2	(1.800000, 2.400000)	2.000000	(1.800000, 2.400000)	2.000000
3	(1.140000, 1.520000)	0.900000	(1.122000, 1.496000)	0.870000
4	(0.837000, 1.116000)	0.395000	(0.810930, 1.081240)	0.351550
5	(0.703350, 0.937800)	0.172250	(0.681855, 0.909141)	0.136426
6	(0.644993, 0.859990)	0.074988	(0.630983, 0.841310)	0.051638
7	(0.619578, 0.826105)	0.032631	(0.611539, 0.815386)	0.019232
8	(0.608518, 0.811358)	0.014197	(0.604251, 0.805669)	0.007086
9	(0.603706, 0.804942)	0.006177	(0.601555, 0.802073)	0.002591
10	(0.601613, 0.802150)	0.002688	**(0.600566, 0.800754)**	**0.000943**
11	(0.600702, 0.800935)	0.001169	(0.600205, 0.800273)	0.000342
12	**(0.600305, 0.800407)**	**0.000509**	(0.600074, 0.800099)	0.000124

Tolerance $\varepsilon = 10^{-3}$: single-step needs $n = 12$ iterations; multi-step needs only $n = 10$ – a 17% reduction, achieved simply by mixing in a small ($\theta_2 = 0.01$) contribution from the second-lag velocity. (Larger θ_2, e.g. 0.05, overshoots into the interior of C and stalls at the *wrong* point of $\mathrm{Fix}(T)$ – see next slide.)

Worked Example: Figures





Left: trajectories in $\mathbb{R}^2$ (single-step vs. multi-step). Right: $\|x_n - x^\star\|$ vs. n on a log scale – the multi-step curve drops below the multi-step curve's single-step counterpart earlier.

A Cautionary Tale: Why θ_1, θ_2 Can't Be Too Large

Since $\mathrm{Fix}(T)$ is the *whole* disk C (every interior point is already fixed by the projection T), overshooting matters: if the extrapolated point y_n ever lands *inside* C, then $T(y_n) = y_n$ exactly, and the update becomes $x_{n+1} = y_n$ with **no further contraction**. With $\theta_1 = 0.15, \theta_2 = 0.05$ (a larger total inertial weight 0.20), y_n enters the disk around $n = 7$ and the sequence *freezes* at $(0.5858, 0.7811)$ – a genuine element of $\mathrm{Fix}(T)$, but *not* $x^\star = (0.6, 0.8)$. This is exactly the failure mode that condition (6.12)/Remark 6.4 warns about: without a decay mechanism, $\sum a_n \|x_n - x_{n-1}\|$ can converge to the *wrong* limit. Smaller, carefully tuned weights ($\theta_1 = 0.13, \theta_2 = 0.01$ above) accelerate convergence *without* triggering this instability.

Python: Multi-step Inertial KM (Section 6.1)

```python
import numpy as np

def T(x):
    """Projection onto the closed unit disk (nonexpansive)."""
    nrm = np.linalg.norm(x)
    return x / nrm if nrm > 1.0 else x.copy()

def multistep_inertial_km(x0, thetas, lam=0.5, n_iter=30):
    """Multi-step inertial KM with S_n = {0,...,s-1}, b_{n,k}=a_{n,k}
    (Remark 6.4 case: z_n = y_n).  thetas = [theta_1,...,theta_s]."""
    s = len(thetas)
    xs = [x0.copy() for _ in range(s + 1)]       # x_{-1}=...=x_{-s}=x0
    for n in range(n_iter):
        yn = xs[-1].copy()
        for k in range(1, s + 1):
            yn = yn + thetas[k - 1] * (xs[-k] - xs[-k - 1])
        x_next = (1 - lam) * yn + lam * T(yn)
        xs.append(x_next)
    return np.array(xs[s:])                      # drop warm-up copies

x0 = np.array([3.0, 4.0])
traj = multistep_inertial_km(x0, thetas=[0.13, 0.01])
print(traj[9])   # -> (0.600566, 0.800754), already within 1e-3 of (0.6,0.8)
```

6.2 Intuition

The problem to solve. The online rule (6.11) is safe but costly: recomputing $a_{n,k}, b_{n,k}$ at every step, for potentially all $k \in \{0, \dots, n-1\}$, means the bookkeeping grows with n. **Can we pick the weights once, in closed form, and never look at $\{x_n\}$ again?** This section gives *two* such pre-scheduled families: (A) weighted averages of *all* past iterates with hand-picked weight arrays $\{\mu_{n,k}\}, \{\nu_{n,k}\}$, and (B) a *recursive* exponential-memory variant that can be updated in $O(1)$ time per step. Both converge *unconditionally* – no summability condition on $\{x_n\}$ is required at all.

(A) General KM on the Affine Hull of Orbits

Definition (6.13)

Let $\lambda \in (0,1)$ and let $\{\mu_{n,k}\}_{n\in\mathbb{N},0\le k\le n}$, $\{\nu_{n,k}\}_{n\in\mathbb{N},0\le k\le n}$ be two real arrays (satisfying technical assumptions (B1)–(B4), notably that each row sums to 1). Take $x_0 \in \mathcal{H}$ and, for each $n \ge 0$,

$$\begin{cases} y_n = \displaystyle\sum_{k=0}^{n} \mu_{n,k}\, x_k, \\ z_n = \displaystyle\sum_{k=0}^{n} \nu_{n,k}\, x_k, \\ x_{n+1} = (1-\lambda) y_n + \lambda\, T(z_n). \end{cases}$$

Symbols. y_n, z_n: weighted averages of *every* past iterate $x_0, \dots, x_n$, using weight rows $\{\mu_{n,k}\}_k$ and $\{\nu_{n,k}\}_k$ that are fixed once, before the algorithm runs, as functions of (n,k) only. Because the weights never reference $\|x_i - x_j\|$, this scheme is unconditional by construction.

(A): Reduction to Multi-step Inertial Form I

Telescoping the sum for y_n using $\sum_{k=0}^{n} \mu_{n,k} = 1$ (assumption (B2)) shows (6.13) is a special case of the general multi-step scheme (6.5):

$$y_n = x_n + (\mu_{n,n} - 1)(x_n - x_{n-1}) + (\mu_{n,n} + \mu_{n,n-1} - 1)(x_{n-1} - x_{n-2}) + \cdots$$

i.e. with $S_n = \{0, \ldots, n-1\}$ and $a_{n,k} = \sum_{j=n-k}^{n} \mu_{n,j} - 1$ (similarly $b_{n,k}$ from ν).
Two useful special cases.

(6.16): only y_n is averaged

$\nu_{n,n} = 1,\ \nu_{n,k} = 0\ (k < n)$, so $z_n = x_n$:

$$y_n = \sum_{k=0}^{n} \mu_{n,k} x_k, \qquad x_{n+1} = (1 - \lambda) y_n + \lambda T(x_n).$$

(A): Reduction to Multi-step Inertial Form II

(6.17): only z_n is averaged

$\mu_{n,n} = 1,\ \mu_{n,k} = 0\ (k < n)$, so $y_n = x_n$:

$$z_n = \sum_{k=0}^{n} \nu_{n,k} x_k, \qquad x_{n+1} = (1-\lambda)x_n + \lambda T(z_n).$$

And if $\nu_{n,k} = \mu_{n,k}$ for all n, k, (6.13) reduces to the earlier (single-array) general inertial KM iteration of Chapter 3.

Example 6.1: (p, q)-step Inertial KM

Idea. Instead of averaging over *all* of $x_0, \dots, x_n$, only average over the last p (for y_n) or last q (for z_n) iterates, using *fixed* weights $\{\mu_i\}_{i=0}^{p}$, $\{\nu_j\}_{j=0}^{q}$ with $\sum_i \mu_i = \sum_j \nu_j = 1$:

$$\mu_{n,k} = \begin{cases} 0, & 0 \le k < n - p, \\ \mu_{n-k}, & n - p \le k \le n, \end{cases} \qquad \text{(similarly for } \nu_{n,k} \text{ with } q\text{)}.$$

This is called the (p, q)-**step inertial KM iteration**: y_n looks back p steps, z_n looks back q steps, with hand-chosen constant weights. By Corollary 6.1 (below), whenever T is nonexpansive with $\mathrm{Fix}(T) \neq \emptyset$, the (p, q)-MiKM sequence converges weakly to a point of $\mathrm{Fix}(T)$ – *unconditionally*, i.e. this is precisely a rigorous version of the "s-step method," now with a convergence guarantee that Ortega–Rheinboldt's original proposal lacked.

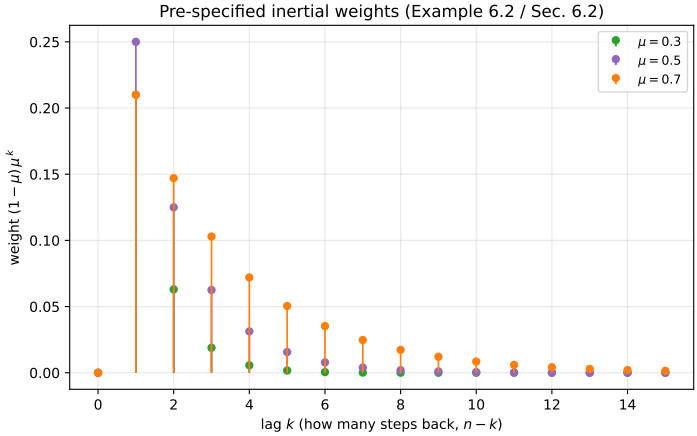
Example 6.2: Exponential Memory Weights

Idea. Instead of a hard window of size p, let the influence of older iterates decay *geometrically*. Fix $\mu, \nu \in (0,1)$:

$$\mu_{n,k} = \begin{cases} \mu^n, & k = 0, \\ (1-\mu)\mu^{n-k}, & 1 \le k \le n, \end{cases} \qquad \nu_{n,k} = \begin{cases} \nu^n, & k = 0, \\ (1-\nu)\nu^{n-k}, & 1 \le k \le n. \end{cases}$$

Reading it. Writing lag $= n - k$ (how far back), the weight on a term x_k is $(1-\mu)\mu^{\text{lag}}$: the most recent iterate gets weight $\approx (1-\mu)$, and each step further back is discounted by another factor of μ – an *exponential moving average* (EMA). This is a bona-fide element of the family (6.13): $\{\mu_{n,k}\}$ and $\{\nu_{n,k}\}$ satisfy the needed structural assumptions (A1)–(A3), (A5), so by Corollary 6.1 the scheme converges. (Whether the *combined* array $(1-\lambda)\mu_{n,k} + \lambda\nu_{n,k}$ also satisfies them for every $\lambda \in (0,1)$ – needed for the sharper Theorem 6.4 – is, interestingly, still **open**.)

Figure: The Fixed Exponential Weight Schedule



The weight $(1-\mu)\mu^k$ assigned to the iterate k steps back, for three choices of the decay parameter μ. Every one of these curves is known completely *before* the algorithm is run – it is a fixed function of the lag k and does not reference $\{x_n\}$ at all.

Corollary 6.1 and Theorem 6.4 I

Theorem 6.4 (general, weighted)

Under structural assumptions (B1)–(B4) on μ, ν, and on $(1-\lambda)\mu_{n,k} + \lambda\nu_{n,k}$ for every $\lambda \in (0,1)$, plus a technical summability condition (6.18) involving an auxiliary sequence $\{\chi_n\}$, the sequence $\{x_n\}$ from (6.13) converges weakly to a point of $\mathrm{Fix}(T)$.

Condition (6.18) is hard to check directly (it still mixes $\{x_n\}$ into the picture). **Good news:** if the arrays are *nonnegative*, this condition can be discharged automatically.

Corollary 6.1 – the clean, unconditional statement

Let $T : \mathcal{H} \to \mathcal{H}$ be nonexpansive with $\mathrm{Fix}(T) \neq \emptyset$. If $\{\mu_{n,k}\}, \{\nu_{n,k}\}$ are *nonnegative* sequences such that $\{\mu_{n,k}\}$, $\{\nu_{n,k}\}$, and $\{(1-\lambda)\mu_{n,k} + \lambda\nu_{n,k}\}$ (for all $\lambda \in (0,1)$) satisfy (A1)–(A3) and (A5), then $\{x_n\}$ generated by (6.13) converges weakly to a point of $\mathrm{Fix}(T)$.

This is the payoff: Examples 6.1 and 6.2 both supply nonnegative arrays satisfying (A1)–(A3),(A5) (Example 6.1 via a known construction, Example 6.2 via [46, Example 2.5]), so *both* the (p, q)-step

Corollary 6.1 and Theorem 6.4 II

scheme and the exponential-memory scheme converge weakly to a point of $\mathrm{Fix}(T)$ – with **zero** bookkeeping tied to the run.

(B) The Modified KM Iteration – A Recursive EMA

Motivation. Example 6.2's weights are exact, but computing y_n from scratch each time still means summing over all of $x_0, \ldots, x_n$ – an $O(n)$ cost per step. Can the *same* exponential-memory idea be computed in $O(1)$ time? Yes: keep running EMAs y_n, z_n and update them recursively.

Definition (6.19)

For $x_0, y_0, z_0 \in \mathcal{H}$, $\lambda \in (0,1)$, $\alpha, \beta \in [0,1]$, and $n \geq 1$:

$$\begin{cases} y_{n+1} = (1-\alpha)x_n + \alpha y_n, \\ z_{n+1} = (1-\beta)x_n + \beta z_n, \\ x_{n+1} = (1-\lambda)y_{n+1} + \lambda T(z_{n+1}). \end{cases}$$

Symbols. α, β: fixed memory-decay constants (bigger $\Rightarrow$ longer memory); y_{n+1}, z_{n+1}: the updated EMAs – each is just a convex combination of the newest iterate and the previous EMA.

(B): Unrolling the Recursion

Unrolling $y_{n+1} = (1-\alpha)x_n + \alpha y_n$ (with $y_0 = x_0$) gives exactly

$$y_{n+1} = \sum_{k=1}^{n} (1-\alpha)\alpha^{n-k} x_k + \alpha^n x_0,$$

i.e. (6.19) with $y_0 = z_0 = x_0$ is the *same* weighted average as Example 6.2 (with $\mu = \alpha, \nu = \beta$) – just computed by carrying one running vector forward instead of re-summing every time.

Key practical point – Remark 6.9

Computing x_{n+1} in (6.19) only ever needs x_n, y_n, z_n – it does *not* need to keep $x_0, x_1, \ldots, x_{n-1}$ in memory at all. This is genuinely cheaper than evaluating the weighted sums of (6.13)/Example 6.2 directly, which do need the full history.

Special cases: $\beta = 0$ gives (6.22) (only y is smoothed); $\alpha = 0$ smooths only z; $\beta = \alpha$ smooths y and z identically; $\alpha = \beta = 0$ recovers the plain KM iteration.

(B): Convergence and Rate – Theorems 6.5, 6.6

Theorem 6.5 – unconditional convergence

If $T : \mathcal{H} \to \mathcal{H}$ is nonexpansive with $\mathrm{Fix}(T) \neq \emptyset$, then $\{x_n\}$ generated by (6.19) converges weakly to a point of $\mathrm{Fix}(T)$ – **for any** fixed $\alpha, \beta \in [0,1]$, $\lambda \in (0,1)$. No summability condition on $\{x_n\}$ is needed.

Theorem 6.6 – running-average complexity bound

$$\frac{1}{n}\sum_{k=0}^{n} \|x_k - T(x_k)\|^2 \leq \frac{3}{n}\Big(\frac{C}{\tau} + \|y_0 - T(z_0)\|^2\Big), \tag{6.23}$$

where $\tau = \min\{\lambda(1-\lambda), (1-\lambda)\alpha, \lambda\beta\}$ and
$C = \inf_{x \in \mathrm{Fix}(T)}\{\|x_0 - x\|^2 + \frac{(1-\lambda)\alpha}{1-\alpha}\|y_0 - x\|^2 + \frac{\lambda\beta}{1-\beta}\|z_0 - x\|^2\}$.

Reading it. The average residual $\|x_k - T(x_k)\|^2$ shrinks like $O(1/n)$, with a constant C/τ that grows if $\alpha, \beta \to 1$ (longer memory, larger constant) or $\lambda \to 0, 1$ (degenerate KM step).

Python: Recursive $O(1)$ Inertial Update (Section 6.2)

```python
import numpy as np

def modified_km_step(x_n, y_n, z_n, alpha, beta, lam, T):
    """One step of the modified KM iteration (6.19): O(1) update,
    parameters alpha, beta, lam are FIXED in advance -- no dependence
    on the run-time sequence {x_n} is needed."""
    y_next = (1 - alpha) * x_n + alpha * y_n
    z_next = (1 - beta) * x_n + beta * z_n
    x_next = (1 - lam) * y_next + lam * T(z_next)
    return x_next, y_next, z_next

def run_modified_km(x0, alpha, beta, lam, T, n_iter=40):
    x_n, y_n, z_n = x0.copy(), x0.copy(), x0.copy()
    hist = [x_n.copy()]
    for _ in range(n_iter):
        x_n, y_n, z_n = modified_km_step(x_n, y_n, z_n, alpha, beta, lam, T)
        hist.append(x_n.copy())
    return np.array(hist)

# alpha, beta play the role of mu, nu in the Example 6.2 weight schedule
traj = run_modified_km(np.array([3.0, 4.0]), alpha=0.5, beta=0.5, lam=0.5,
                       T=lambda x: x / max(np.linalg.norm(x), 1.0))
```

6.3 Intuition

Every operator-splitting method for $\operatorname{zer}(A+B)$ or a monotone inclusion that is built out of a nonexpansive map T can be "inertialized" by the same recipe: replace the input to T (and to the resolvent $J_{\gamma A}$ feeding it) with an inertially extrapolated point y_n or z_n built from $\sum_{k\in S_n} a_{n,k}(x_{n-k}-x_{n-k-1})$, exactly as in (6.5). This section walks through five such constructions – forward-backward, backward-forward, primal-dual, Douglas-Rachford, and Davis-Yin splitting – each now with multi-step memory.

(A) Multi-step Inertial Forward-Backward Splitting

Problem. Find $\operatorname{zer}(A+B)$: e.g. minimize $f(x)+g(x)$ where $A=\partial f$ is the subdifferential of a nonsmooth convex regularizer (such as $f(x)=\mu\|x\|_1$, giving LASSO-type sparsity) and $B=\nabla g$ is the gradient of a smooth, β-cocoercive loss (such as $g(x)=\frac{1}{2}\|Mx-b\|^2$). Then $J_{\gamma A}=\operatorname{prox}_{\gamma f}$.

Definition (6.24)

Let $\gamma\in(0,2\beta)$, $\{\lambda_n\}\subset[0,\frac{4\beta-\gamma}{2\beta}]$ with $\sum_n \lambda_n(\frac{4\beta-\gamma}{2\beta}-\lambda_n)=\infty$:

$$\begin{cases} y_n = x_n + \displaystyle\sum_{k\in S_n} a_{n,k}(x_{n-k}-x_{n-k-1}), \\ z_n = x_n + \displaystyle\sum_{k\in S_n} b_{n,k}(x_{n-k}-x_{n-k-1}), \\ x_{n+1} = (1-\lambda_n)y_n + \lambda_n J_{\gamma A}(\mathrm{Id}-\gamma B)z_n. \end{cases}$$

Theorem 6.7. Under condition (6.25) (the (6.9)-type summability condition), $\{x_n\}$ weakly converges to a point of $\operatorname{zer}(A+B)$.

(B) Multi-step Inertial Backward-Forward Splitting

Problem. Same $\operatorname{zer}(A+B)$ target, but resolve A *first*, then take a forward (gradient) step in B – useful when the resolvent of A is cheap to invert but you would rather not compose it with B directly.

Definition (6.26)

$$\begin{cases} u_n = x_n + \displaystyle\sum_{k\in S_n} a_{n,k}(x_{n-k} - x_{n-k-1}), \\ v_n = x_n + \displaystyle\sum_{k\in S_n} b_{n,k}(x_{n-k} - x_{n-k-1}), \\ y_n = J_{\gamma A}(v_n), \qquad z_n = y_n - \gamma B y_n, \\ x_{n+1} = u_n + \lambda_n(z_n - u_n). \end{cases}$$

Theorem 6.8. With $\delta = \min\{1, \beta/\gamma\} + \frac{1}{2}$, if $\{\lambda_n\} \subset [0, \delta]$ satisfies $\sum_n \lambda_n(\delta - \lambda_n) = \infty$ and the (6.9)-type condition holds, $\{x_n\}$ (equivalently $\{y_n\}$) weakly converges to a point of $\operatorname{zer}(A+B)$.

(C) Multi-step Inertial Primal-Dual Splitting

Problem. A saddle-point / monotone-inclusion system coupling a primal variable x and a dual variable v through a bounded linear operator $L : \mathcal{H} \to \mathcal{G}$ – e.g. TV-regularized imaging or constrained optimization with Lx appearing inside a nonsmooth penalty. Here A, C are maximally monotone, B is β_B-cocoercive, D is β_D-strongly monotone, and γ_A, γ_C satisfy

$$2\min\{\beta_B, \beta_D\} \min\left\{\frac{1}{\gamma_A}, \frac{1}{\gamma_C}\right\}\left(1 - \sqrt{\gamma_A \gamma_C \|L\|^2}\right) > 1. \tag{6.27}$$

Definition (6.28) – iteration

$$\begin{cases} y_n = x_n + \sum_{k\in S_n} a_{n,k}(x_{n-k} - x_{n-k-1}), \quad u_n = x_n + \sum_{k\in S_n} b_{n,k}(v_{n-k} - v_{n-k-1}), \\ x_{n+1} = J_{\gamma_A A}(y_n - \alpha B(y_n) - \gamma_A L^* u_n), \qquad \bar{x}_{n+1} = 2x_{n+1} - y_n, \\ v_{n+1} = J_{\gamma_C C^{-1}}\big(u_n - \gamma_C D^{-1}(u_n) + \gamma_C L\bar{x}_{n+1}\big). \end{cases}$$

Theorem 6.9. Under (6.27) and (6.9)-type conditions on $a_{n,k}$ (applied to the x-sequence) and $b_{n,k}$ (applied to the v-sequence), $\{x_n\}$ and $\{v_n\}$ weakly converge to primal/dual solutions.

(D) Multi-step Inertial Douglas-Rachford Splitting

Problem. $\mathrm{zer}(A_1 + A_2)$ via the reflected resolvent operator $R_{\gamma A_1} R_{\gamma A_2}$, where $R_{\gamma A} = 2J_{\gamma A} - \mathrm{Id}$ – e.g. convex feasibility between two sets C_1, C_2 (take $A_i =$ normal cone of C_i).

Definition (6.30)

With $\gamma > 0$, $\{\lambda_n\} \subset [0, 2]$, $\sum_n \lambda_n(2 - \lambda_n) = \infty$:

$$\begin{cases} u_n = x_n + \displaystyle\sum_{k \in S_n} a_{n,k}(x_{n-k} - x_{n-k-1}), \\ v_n = x_n + \displaystyle\sum_{k \in S_n} b_{n,k}(x_{n-k} - x_{n-k-1}), \\ x_{n+1} = \left(1 - \dfrac{\lambda_n}{2}\right) u_n + \dfrac{\lambda_n}{2} R_{\gamma A_1} R_{\gamma A_2}(v_n). \end{cases}$$

Theorem 6.10. Under the (6.9)-type condition, $\{x_n\}$ converges weakly to a point of $\mathrm{Fix}(R_{\gamma A_1} R_{\gamma A_2})$ (from which a zero of $A_1 + A_2$ is recovered via $J_{\gamma A_2}$).

(E) Multi-step Inertial Davis-Yin Splitting

Problem. $\operatorname{zer}(A_1 + A_2 + B)$, a *three*-operator sum – e.g. minimize $f_1(x) + f_2(x) + g(x)$ with two nonsmooth terms ($A_1 = \partial f_1$, $A_2 = \partial f_2$) plus one smooth, cocoercive term ($B = \nabla g$).

Definition (6.32)

$$\begin{cases} u_n = x_n + \sum_{k \in S_n} a_{n,k}(x_{n-k} - x_{n-k-1}), \qquad v_n = x_n + \sum_{k \in S_n} b_{n,k}(x_{n-k} - x_{n-k-1}), \\ y_n = J_{\gamma A_2} v_n, \\ z_n = J_{\gamma A_1}(2y_n - v_n - \gamma B y_n), \\ x_{n+1} = (1 - \lambda_n)u_n + \lambda_n(z_n - y_n + v_n). \end{cases}$$

Theorem 6.11. Under a condition on γ, λ_n and a (6.9)-type summability of $a_{n,k}$, the weak limit x^* of $\{x_n\}$ satisfies $J_{\gamma A_2}(x^*) \in \operatorname{zer}(A_1 + A_2 + B)$.

Python: Multi-step Inertial Forward-Backward (LASSO)

```python
import numpy as np

def soft_threshold(v, t):
    """prox of t*||.||_1 -- the resolvent J_{gamma A} for A = subdiff |.|_1."""
    return np.sign(v) * np.maximum(np.abs(v) - t, 0.0)

def multistep_inertial_fb(M, b, mu, gamma, thetas, n_iter=200):
    """Minimize 0.5*||Mx-b||^2 + mu*||x||_1 via (6.24) with
    b_{n,k}=a_{n,k} (so z_n = y_n), S_n = {0,...,s-1}."""
    d = M.shape[1]
    x = [np.zeros(d) for _ in range(len(thetas) + 1)]   # warm start
    for _ in range(n_iter):
        yn = x[-1].copy()
        for k, th in enumerate(thetas, start=1):
            yn = yn + th * (x[-k] - x[-k - 1])           # multi-step term
        grad = M.T @ (M @ yn - b)                         # B = grad g
        x_next = soft_threshold(yn - gamma * grad, gamma * mu)  # J_{gamma A}
        x.append(x_next)
    return x[len(thetas):]

rng = np.random.default_rng(0)
M = rng.standard_normal((40, 100))
x_true = np.zeros(100); x_true[:5] = rng.standard_normal(5)
b = M @ x_true
sol = multistep_inertial_fb(M, b, mu=0.1, gamma=0.9 / np.linalg.norm(M, 2)**2,
                            thetas=[0.13, 0.01])
```

Chapter 6 Summary

- **6.1**: The multi-step inertial KM iteration (6.5) uses a weighted combination of several lagged displacements $x_{n-k} - x_{n-k-1}$, $k \in S_n$, generalizing every scheme from Chapter 5. Weak convergence holds under summability condition (6.9)/(6.10), which in the single-lag case (Remark 6.4, $b_n = a_n$) reads $\sum_n a_n \|x_n - x_{n-1}\| < \infty$ – a condition that couples the parameters to the (unknown, in advance) behavior of $\{x_n\}$.
- **6.2**: Two ways to *pre-schedule* the inertial weights so they never reference $\{x_n\}$: (A) weighted averages over a window or with exponential decay (Examples 6.1-6.2, via (6.13)); (B) a recursive $O(1)$ exponential-moving-average update (6.19) with an *unconditional* convergence guarantee (Theorem 6.5) and an explicit $O(1/n)$ complexity bound (Theorem 6.6).
- **6.3**: The same multi-step inertial recipe upgrades five classical operator-splitting methods – forward-backward, backward-forward, primal-dual, Douglas-Rachford, Davis-Yin – each converging to a zero of the corresponding monotone-operator sum under a matching summability condition.